Homework 3

Exercise 1

Part (i)

In[1]:=
```
(* Func draws from standard normal N(0,1) using Box-Muller*)

drawStandardNormal[] := Module[{Y},

  (* Box-Muller method *)

  Y =  Sqrt[-2 Log[RandomReal[]]] * Sin[2 * Pi * RandomReal[]];

  Y
 ]
```

In[2]:=
```
(* Func draws from the given bivariate normal N_2(b,(4 -1, -1 4)) *)
drawBivariateNormal[b_, delta_] := Module[{X},

  Y = Table[drawStandardNormal[], 3];

  X = b + Sqrt[delta] {Sqrt[3] * Y[[1]] + Y[[3]], Sqrt[3] * Y[[2]] - Y[[3]]};

  X
 ]
```

In[3]:=
```
(* Func uses R replications in CMC to estimate z=P(X in A_a) with a 95% CI*)
doCrudeMonteCarlo[a_, R_] := Module[{z, CI},

  (* Generate R replicates of Z=1_A(X) *)
  X = Table[drawBivariateNormal[0, 1], R];

  indicatorFunc[{x1_, x2_}] := Boole[x1 ≥ a && x2 ≥ a];

  Z = indicatorFunc /@ X;

  (* Estimate z *)
  z = Mean[Z];

  (* Calculate the CI *)
  s = Sqrt[Variance[Z]];

  CI = {z - 1.96 * s / Sqrt[R], z + 1.96 * s / Sqrt[R]};

  {z, CI}
 ]
```

In[4]:=
```
doCrudeMonteCarlo[1, 100 000]
doCrudeMonteCarlo[3, 100 000]
doCrudeMonteCarlo[10, 100 000]
```

Out[4]= $\left\{\frac{6441}{100\,000}, \{0.0628885, 0.0659315\}\right\}$

Out[5]= $\left\{\frac{3}{2000}, \{0.00126013, 0.00173987\}\right\}$

Out[6]= {0, {0., 0.}}

Part (ii)

In[7]:=
```
Minimize[{2 x1^2 - x1 x2 + 2 x2^2, x1 ≥ a, x2 ≥ a}, {x1, x2}]
```
minimiere

Out[7]= $\left\{\begin{cases} 3a^2 & a > 0 \\ 0 & \text{True} \end{cases}, \left\{x1 \to \begin{cases} a & a > 0 \\ 0 & \text{True} \end{cases}, x2 \to \begin{cases} \frac{1}{4}\left(a + 3\sqrt{a^2}\right) & a > 0 \\ 0 & \text{True} \end{cases}\right\}\right\}$

In[8]:=
```
(* Likelihood ratio function *)
L1[a_, x_] := Exp[-{a, a}.Inverse[{{4, -1}, {-1, 4}}].x +
    1/2 * {a, a}.Inverse[{{4, -1}, {-1, 4}}].{a, a}];
```
Likelihood-Funktion; Exponentialfun··; inverse Matrix; inverse Matrix

In[9]:=
```
(* Func estimates z with importance sampling of R
 replicates where importance distribution is N((a,a),C) *)
doImportanceSampling[a_, R_] := Module[{z, CI},
  (* Generate R replicates of Z=1_A(X)L(X) *)
  X = Table[drawBivariateNormal[{a, a}, 1], R];
  indicatorAndLFunc[{x1_, x2_}] := Boole[x1 ≥ a && x2 ≥ a] * L1[a, {x1, x2}];
  Z = indicatorAndLFunc /@ X;

  (* Estimate z *)
  z = Mean[Z];
  (* Calculate the CI *)
  s = Sqrt[Variance[Z]];
  CI = {z - 1.96 * s / Sqrt[R], z + 1.96 * s / Sqrt[R]};
  {z, CI}
 ]
```
numeris··; Konstante; Modul; Tabelle; charakteristische Boolesche Funktion; arithmetisches Mittel; Qua··; Varianz; Quadratwurzel; Quadratwurzel

In[10]:=
```
doImportanceSampling[1, 100000]
doImportanceSampling[3, 100000]
doImportanceSampling[10, 100000]
```

Out[10]= {0.0645238, {0.063652, 0.0653956}}

Out[11]= {0.00136031, {0.00133264, 0.00138798}}

Out[12]= $\{1.17023 \times 10^{-17}, \{1.10825 \times 10^{-17}, 1.23221 \times 10^{-17}\}\}$

Part (iii)

In[13]:=
```
(* Likelihood ratio function *)
L2[a_, x_, delta_] := Module[{L},
  invC = Inverse[{{4, -1}, {-1, 4}}];
  b = {a, a};
  L = delta * Exp[
     -1 / 2 x.invC.x + 1 / (2 delta) x.invC.b - 1 / delta x.invC.b + 1 / (2 delta) b.invC.b];
  L
 ]
```

In[14]:=
```
(* Func estimates z with importance sampling of R
 replicates where importance distribution is N((a,a),delta C)*)
doImportanceSampling2[a_, R_, delta_] := Module[{z, CI},
  (* Generate R replicates of Z=1_A(X)L(X) *)
  X = Table[drawBivariateNormal[{a, a}, delta], R];
  indicatorAndLFunc[{x1_, x2_}] := Boole[x1 ≥ a && x2 ≥ a] * L2[a, {x1, x2}, delta];
  Z = indicatorAndLFunc /@ X;

  (* Estimate z *)
  z = Mean[Z];
  (* Calculate the CI *)
  s = Sqrt[Variance[Z]];
  CI = {z - 1.96 * s / Sqrt[R], z + 1.96 * s / Sqrt[R]};
  {z, CI}
 ]
```

In[15]:=
```
doImportanceSampling2[1, 100000, 1]
doImportanceSampling2[3, 100000, 1]
doImportanceSampling2[10, 100000, 1]
```

Out[15]= {0.0651362, {0.064258, 0.0660144}}

Out[16]= {0.00138812, {0.00135993, 0.00141631}}

Out[17]= $\{1.17655 \times 10^{-17}, \{1.11393 \times 10^{-17}, 1.23917 \times 10^{-17}\}\}$

In[18]:=
```
doImportanceSampling2[10, 100000, 1]
```

Out[18]= $\{1.1555 \times 10^{-17}, \{1.09352 \times 10^{-17}, 1.21749 \times 10^{-17}\}\}$

In[19]:=
```
result = Table[doImportanceSampling2[10, 100, delta], {delta, 0.5, 100, 0.5}];
```
Tabelle
```
Plot[doImportanceSampling2[10, 100, delta], {delta, 0.5, 10}]
```
stelle Funktion graphisch dar

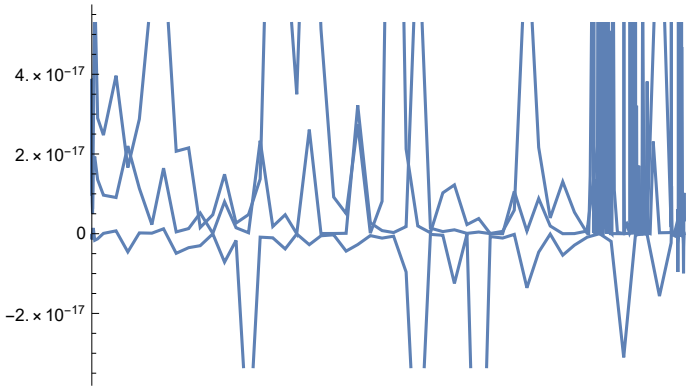
General: Exp[−813.821] is too small to represent as a normalized machine number; precision may be lost.

General: Exp[−741.313] is too small to represent as a normalized machine number; precision may be lost.

General: Exp[−887.745] is too small to represent as a normalized machine number; precision may be lost.

General: Further output of General::munfl will be suppressed during this calculation.

Out[20]=



Exercise 3

Part 1

In[21]:=
```
(* Func does CMC with R replications *)
doCMC[R_] := Module[{z},
                   Modul
  z = Mean[Table[
       arit··  Tabelle
     Boole[Product[RandomReal[], {i, 20}] ≥ Exp[-15 / (0.5 + RandomReal[])]], R]];
      chara··  Produkt  reelle Zufallszahl  Exponentialfunktion  reelle Zufallszahl
  z
 ]
N[doCMC[1000]]
 numerischer Wert
```

Out[22]= 0.298

In[23]:=
```
varCMC = N[Variance[Table[doCMC[1000], 100]]]
          ··  Varianz  Tabelle
```

Out[23]= 0.000223687

Part 2

In[24]:=
```
(* Func generates Z and a control variate W *)
generateZandW[] := Module[{Z, W},
                          Modul
  W = Sum[Log[RandomReal[]], {i, 20}];
      s···  L···  reelle Zufallszahl
  Z = CDF[UniformDistribution[{0, 1}], -15 / W - 0.5];
      k···  Gleichverteilung
  {Z, W}
 ]
```

In[25]:=
```
(* Func provides the control variate estimator with R replications *)
simulateControlVariate[R_] := Module[{ZandW, Z, W, estimate},
  ZandW = Table[generateZandW[], R];
  Z = ZandW[[All, 1]];
  W = ZandW[[All, 2]];
  estimate = Mean[Z] - Covariance[Z, W] / Variance[W] (Mean[W] + 20);
  estimate
 ]
simulateControlVariate[1000]
```

Out[26]= 0.292101

In[27]:=
```
varControlVariateSim = N[Variance[Table[simulateControlVariate[1000], 100]]]
```

Out[27]= 4.20094×10^{-6}

Part 3

In[28]:=
```
(* generates vector [Z_1,...,Z_R/2, ..., Z_R] with antithetic second half *)
generateZ[] := Module[{U, Z, antitheticZ},
  U = Table[RandomReal[{0, 1}], 20];
  Z = CDF[UniformDistribution[{0, 1}], -15 / Sum[Log[U[[i]]], {i, 20}] - 0.5];
  antitheticZ =
   CDF[UniformDistribution[{0, 1}], -15 / Sum[Log[1 - U[[i]]], {i, 20}] - 0.5];
  {Z, antitheticZ}
 ]
```

In[29]:=
```
(* simulates using antithetic variables *)
antitheticSimulation[R_] := Module[{z},
  z = Mean[Mean[Table[generateZ[], R / 2]]];
  z
 ]
antitheticSimulation[1000]
```

Out[30]= 0.283359

In[31]:=
```
varAntithetic = N[Variance[Table[antitheticSimulation[1000], 100]]]
```

Out[31]= 0.0000137377

Exercise 5

Control Variate

In[32]:=
```
X = Table[RandomVariate[ExponentialDistribution[1]], 10];
Z = Table[(x + 0.02 x^2) Exp[0.1 Sqrt[1 + Cos[x]]] , {x, X}];
W = Table[(x + 0.02 x^2) Exp[0.1], {x, X}];
Correlation[Z, W]
```

Out[35]=
```
0.998759
```

In[36]:=
```
Integrate[(x + 0.02 x^2) Exp[0.1 - x], {x, 0, Infinity}]
```

Out[36]=
```
1.14938
```

In[37]:=
```
Plot[(x + 0.02 x^2) Exp[0.1 Sqrt[1 + Cos[x]]], {x, 0, 100}]
```

Out[37]=